

1. Informações Gerais

* **Título do Jogo:** **Cerco Esquelético**

* **Gênero:** Ação 2D / Plataforma / Defesa de Objetivo (Escort/Tower Defense)

* **Motor Gráfico (Engine):** Unity 2D

2. Visão Geral do Jogo

"Realidade Bugada" é um jogo de ação lateral em 2D onde o jogador assume o papel de um guerreiro. O objetivo principal das fases gira em torno de combater hordas de mortos-vivos (esqueletos) para proteger um alvo específico, muito provavelmente uma garota, de ataques contínuos.

3. Personagens e Entidades

O projeto é composto por três entidades principais:

* **Jogador (Player / Warrior):**

* **Visual:** Utiliza um sprite sheet de guerreiro (`Warrior\_Sheet-Effect.png`).

* **Ações (Animações):** O jogador possui um controlador completo de movimentação de plataforma e combate, incluindo animações de repouso (`PlayerIdle.anim`), movimentação (`PlayerMove.anim`), pulo (`PlayerJump.anim`), queda (`PlayerFall.anim`) e ataque (`PlayerAttack.anim`).

* **Scripts:** Controlado pelo script `Player.cs`.

* **Inimigo (Enemy / Skeleton):**

* **Visual:** Representado por sprites de esqueletos (`Skeleton Idle.png`, `Skeleton Walk.png`, `Skeleton Attack.png`).

* **Ações (Animações):** Movimenta-se em direção ao alvo e realiza ataques corporais (`EnemyIdle.anim`, `EnemyMove.anim`, `EnemyAttack.anim`).

* **Scripts:** Gerenciado pelo script `Enemy.cs`. O jogo conta com um sistema de geração contínua de inimigos através do script `EnemyRespawn.cs`.

* **O Alvo (Girl / Object To Protect):**

* **Visual:** Representada graficamente por uma garota (`GirlGraphic.png`) com uma animação básica de repouso (`girl\_idle.anim`).

* **Mecânica:** Esta entidade está ligada ao script `ObjectToProtect.cs`, indicando que é a condição de vitória/derrota do jogo. Se ela for destruída pelos inimigos, o jogador perde.

4. Mecânicas de Jogo (Gameplay)

* **Combate Hack and Slash 2D:** O jogador precisa se movimentar e usar seus ataques corpo a corpo para destruir as ondas de inimigos antes que eles alcancem o alvo.

* **Feedback de Combate:** O jogo possui um sistema visual para quando as entidades recebem dano, evidenciado pelo material `DamageFeedback.mat`.

* **Interações Físicas:** O uso de materiais físicos customizados, como o `Sticky.physicsMaterial2D`, indica que o jogo pode ter mecânicas de plataforma onde o jogador adere a certas paredes ou superfícies.

5. Level Design e Mundo

* **Cenários:** Os níveis são construídos utilizando múltiplas camadas de fundo (do `Layer\_0011\_0` ao `Layer\_0000\_9`), o que cria um efeito de profundidade (parallax) rico visualmente. Também há sprites específicos para iluminação (`Layer\_0004\_Lights.png`, `Layer\_0007\_Lights.png`).

* **Cenas (Fases):** O projeto está dividido em pelo menos três cenas:

* `Menu.unity`: A tela inicial, controlada pelo script `MenuPrincipal.cs`.

* `SampleScene.unity` e `SampleScene1.unity`: Fases jogáveis.

6. Interface de Usuário (UI)

* A interface e o HUD do jogo são gerenciados pelo script `UI.cs`.

* Os textos e menus são renderizados usando o pacote **TextMesh Pro**, utilizando uma variedade de fontes estilizadas e dinâmicas (como *Anton*, *Bangers*, *Oswald* e *Roboto*).